

RoadAssist Bharat — Review-Based Work Plan

4 members · 3 reviews · 18 weeks — "One Platform. Every Vehicle. Every Phone. Every Road."

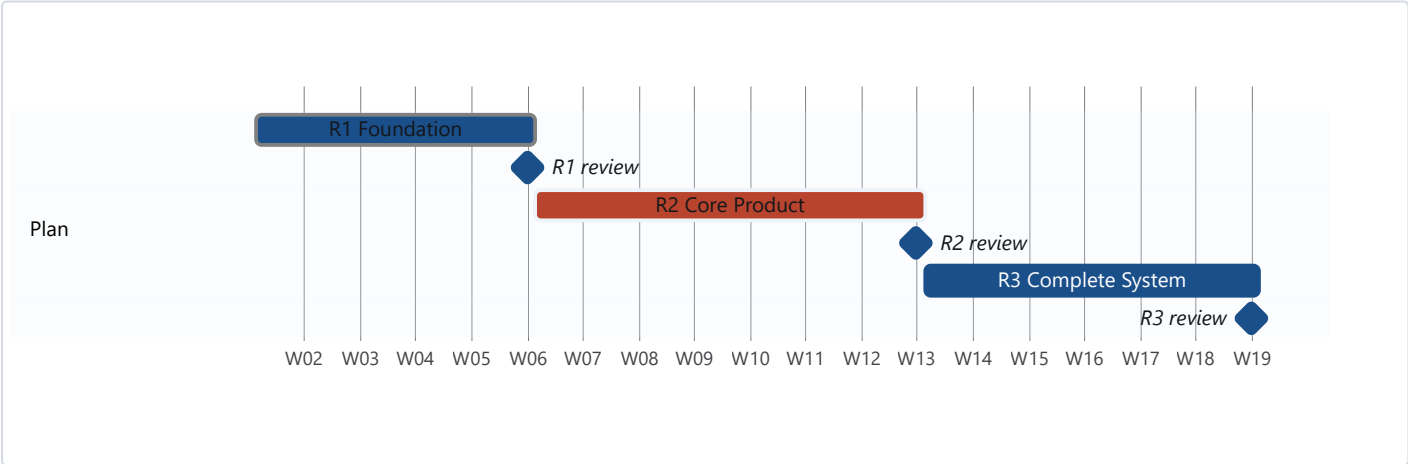
1. The Team

	Role	Owns end-to-end	Total effort
P1	Backend & Database Lead	Server services, PostgreSQL/PostGIS schema, all API contracts, authentication + RBAC, booking & dispatch engine, emergency core, payments, sync server	87 days
P2	Frontend & Mobile Lead	React Native app, web + government + admin portals, design system, offline client, maps UI, accessibility, 8 languages	86 days
P3	AI & Data Lead	9 AI subsystems, model training + serving, on-device inference, feature store, voice/NLU, analytics pipeline	70 days
P4	DevOps, QA & Security Lead	Cloud infrastructure, CI/CD, observability, all testing, security, SMS/IVR/USSD telecom gateway, deployment, compliance	76 days

Backup pairs (so no single point of failure): P1 ↔ P3 · P2 ↔ P4 Each member reviews the other's code. Nobody merges their own work.

2. Review Calendar

Review	Weeks	Duration	Theme	Completion
Review 1	1 – 5	5 weeks	Foundation — research, design, database, login working	25%
Review 2	6 – 12	7 weeks	Core Product — full booking journey, AI, offline, maps	70%
Review 3	13 – 18	6 weeks	Complete System — emergency, government, testing, deployment	100%



3. Effort Distribution

Member	Review 1 (5 wk)	Review 2 (7 wk)	Review 3 (6 wk)	Total
P1 Backend	25 days	33 days	29 days	87
P2 Frontend	17 days	36 days	33 days	86
P3 AI	13 days	33 days	24 days	70
P4 DevOps/QA	17 days	27 days	32 days	76

Member	Review 1 (5 wk)	Review 2 (7 wk)	Review 3 (6 wk)	Total
Team total	72 days	129 days	118 days	319
Per person per week	3.6 days	4.6 days	4.9 days	

Review 1 is deliberately light. Design work that is rushed costs three times as much to undo later. The load rises as the design stabilises.

Two rebalancing decisions were made so nobody is overloaded: 1. P4 takes the **Admin console and Fleet dashboard** off P2 in Review 2 (P2 would otherwise be at 6.3 days/week). 2. P1, P2 and P3 each absorb **2 days of test execution** from P4 in Review 3 (P4 would otherwise be at 6.0 days/week).

P2 remains the tightest at 5.1 days/week in Review 2. Mitigation: the **Android Auto prototype is the declared first cut** — dropping it releases 2 days without touching any graded deliverable.

REVIEW 1 — Foundation

Weeks 1–5 · 72 team-days · 25% complete

Goal: prove the problem is real, the design is sound, and the system's skeleton runs. No feature is complete yet — and that is correct at this stage.

What the panel will see

- Field evidence** — 20 vehicle owners, 10 mechanics, 2 fleet managers interviewed. Real quotes, not assumptions.
- Live database** — migrations run on an empty database, 100,000 realistic rows seeded in under 2 minutes, geospatial "nearest mechanic" query returning in under 50 ms.
- Working login** — OTP arrives on a real phone, user logs in, token refreshes automatically. Then a stolen token is replayed and the system revokes the entire session family.
- Clickable prototype** — the complete breakdown journey as a click-through, in Hindi.
- CI pipeline** — a pull request with a bug, a failing test and a hardcoded password is blocked automatically in under 8 minutes.

Work split

	P1 — Backend	P2 — Frontend	P3 — AI	P4 — DevOps/QA
Weeks 1–2	Integration feasibility study (VAHAN, ERSS 112, UPI); candidate entity list; backend backlog; monorepo scaffold	Device + connectivity study; 20 user interviews; 6 personas; 4 journey maps; wireframes for 12 screens	Data availability audit; model feasibility (which of the 9 are real in v1); labelling plan	Cloud + compliance research; start TRAI DLT registration (longest lead item); CI skeleton
Weeks 3–4	C4 architecture + 15 ADRs ; API style guide; 60-table schema , indexes, audit log, soft delete	Information architecture; design system foundations; app scaffolds; local (offline) schema	AI Gateway design; feature store schema; label capture design	STRIDE threat model; SLO definitions; database infrastructure + backups
Week 5	Authentication — OTP, JWT + rotating refresh, RBAC engine, consent ledger	Auth screens, secure token storage, silent refresh, consent centre	Fraud signal catalogue	Vault secrets, SMS gateway with 2 vendors, edge rate limiting, WAF
Days	25	17	13	17

Acceptance criteria

#	Criterion	Owner
1	Every product assumption is backed by field evidence or explicitly marked as an open bet	All
2	Migrations apply and roll back cleanly on an empty database, twice in a row	P1
3	Every table has a primary key, timestamps, soft delete, version column and an owning service	P1
4	Zero sequential scans on the top-20 queries at 100,000 rows	P1
5	OTP brute force blocked at 5 attempts / 15 minutes; refresh token reuse revokes the whole family	P1

#	Criterion	Owner
6	20 RBAC policy tests pass, including 6 negative access-control cases	P1
7	Wireframes cover loading, empty, error and offline states for all 12 screens	P2
8	Design system renders correctly in all 8 languages with no text truncation	P2
9	Each of the 9 AI systems is classified honestly: trainable now / trainable later / rules-only in v1	P3
10	Every model has a baseline recorded before training begins	P3
11	CI blocks lint errors, failing tests and secrets in under 8 minutes	P4
12	Every regulatory obligation is mapped to the phase that satisfies it	P4

REVIEW 2 — Core Product

Weeks 6–12 · 129 team-days · 70% complete

Goal: a person can use this. The complete breakdown journey works on a real phone, on a real network — and keeps working when the network disappears.

What the panel will see

- A full booking, end to end** — request help → AI diagnosis → 3 mechanics offered → one accepts → live tracking on a map → work completed → UPI payment → rating.
- The offline demo (the differentiator)** — phone in **airplane mode**: add a vehicle, run a diagnosis using the on-device model, create a help request, view an offline map. Reconnect. Everything syncs, one deliberate conflict is raised and resolved.
- The feature-phone demo** — on an actual ₹1,200 phone with no internet: send **MADAD** by SMS, receive the reply, confirm location, get the mechanic's details, cancel. Then the same in Tamil over an IVR call.
- AI with the plug pulled** — the AI service is switched off completely, mid-demo. The product keeps working on rule-based fallbacks. Nothing breaks.
- A real diagnosis** — photograph an actual damaged part, get the fault, severity, parts list and a "do not drive this" verdict in Hindi.

Work split

	P1 — Backend	P2 — Frontend	P3 — AI	P4 — DevOps/QA
Weeks 6–8	140 API endpoints ; booking state machine; dispatch engine; pricing; UPI payments; webhooks	Design system + 48 screens ; onboarding, garage, request-help flow, live tracking, payment	AI Gateway live with rule-based logic for all 8 capabilities (so nothing waits on models)	Observability stack (metrics, logs, traces); preview environments; mobile CI with signed builds
Weeks 9–10	AI client with circuit breakers + fallback registry ; feature pipeline; BFF composite endpoints	Mechanic app; web PWA; AI confidence UI; user feedback loop	7 models trained — diagnosis, image, voice, maintenance, matching, crash, fraud	ML infrastructure; GPU cost controls; Admin console + Fleet dashboard (taken off P2)
Weeks 11–12	Sync server — change feed, conflict resolution, tombstones, replay; geofencing; diagnostics service	Offline client — local DB, operation queue, sync engine, conflict UI; maps + offline packs; OBD Bluetooth	On-device model bundle ≤ 15 MB ; vehicle-class rule packs; EV diagnostics	Telecom gateway — SMS commands, IVR in 8 languages, USSD; tile server; network chaos harness
Days	33	36	33	27

Acceptance criteria

#	Criterion	Owner
1	100% of the v1 API implemented; specification generated from code, never hand-written	P1
2	Every write endpoint is idempotent — proven by a replay test	P1

#	Criterion	Owner
3	The booking state machine rejects every illegal transition (exhaustive test)	P1
4	Complete booking journey works on Android 10 / 2 GB RAM / throttled 3G	P2
5	Every screen has loading, empty, error and offline states built — verified in review	P2
6	Airplane-mode journey completes and syncs with zero data loss	P2 + P1
7	30-scenario conflict test suite passes; no scenario produces a corrupt state	P1 + P2
8	Every model beats its recorded baseline on a held-out test set	P3
9	On-device inference under 120 ms on the low-end reference phone	P3
10	Diagnosis never wrongly says "safe to drive" — measured false-safe rate is zero	P3
11	Kill the AI service entirely: the full booking still completes on fallbacks	P1 + P3
12	A feature-phone user can request help, track and cancel entirely over SMS	P4
13	Navigation and search work with the device fully offline	P2 + P4
14	Sync of 100 queued operations completes in under 8 seconds on 3G	P1 + P2

REVIEW 3 — Complete System

Weeks 13–18 · 118 team-days · 100% complete

Goal: the life-safety features work, the system is tested under failure, and it is deployed and running in production.

What the panel will see

- Crash to responder** — a simulated crash on a real phone. A 30-second countdown takes over the locked screen with alarm and vibration. It completes. Emergency contacts receive an SMS with a live location link, the nearest responder is notified, the incident room opens. **Under 10 seconds.**
- The proof that it is engineered, not assembled** — the entire platform is scaled to **zero replicas**, live, in front of the panel. An SOS is triggered. It still works, over the isolated degraded path.
- False-positive proof** — driving over a real speed breaker on camera. Nothing happens. Measured over 200 hours of real driving.
- Government dashboard** — log in as a district officer, see live incidents in that jurisdiction only, identify an accident blackspot, generate the statutory monthly report as a PDF. Then **attempt to identify an individual citizen and fail** — with the audit log showing the attempt.
- Accessibility proof** — the complete booking journey driven by a screen reader **with the display switched off**.
- Deployment** — one commit reaches production through the full pipeline. Then a broken version is deployed on purpose and rolls back automatically in under 2 minutes.

Work split

	P1 — Backend	P2 — Frontend	P3 — AI	P4 — DevOps/QA
Weeks 13–14	Emergency engine — escalation ladder, ERSS 112 handoff, break-glass medical access, degraded SMS path	SOS UI + crash countdown + incident room; government portal with live map and 6 statutory reports	Crash detection tuned to production; severity model; blackspot detection; incident forecasting	Isolated emergency infrastructure; chaos test proving SOS survives total platform failure ; mTLS for government
Weeks 15–16	Anonymization with k-anonymity ≥ 10 ; CDC pipeline; property-based + mutation testing	Analytics dashboards; fleet + mechanic screens; Detox end-to-end tests ; device farm; accessibility + 8-language QA	ClickHouse analytics + metric dictionary ; bias audit across all models; adversarial testing	Load testing (10k requests/sec), chaos, security testing ; bug burn-down to zero critical issues
Weeks 17–18	Online migrations; health probes; query tuning, caching; scaling plan + tech debt register	App store submission; over-the-air updates; APK size, startup, jank and data-usage optimisation	Model serving + automated rollback; inference optimisation; AI roadmap and data flywheel	Terraform production, ArgoCD, disaster-recovery drill ; infrastructure right-sizing; cost model; compliance roadmap

	P1 — Backend	P2 — Frontend	P3 — AI	P4 — DevOps/QA
Days	29	33	24	32

Acceptance criteria

#	Criterion	Owner
1	Crash → contacts and responders notified in under 10 seconds (95th percentile)	P1
2	Emergency path survives a chaos test with every other service down	P4
3	The AI cannot auto-dispatch emergency services — enforced in code, not policy	P1 + P3
4	Crash false-positive rate ≤ 1 per 1,000 driving hours, measured over real driving	P3
5	Break-glass medical access logs the actor and reason, and notifies the user afterwards	P1
6	No citizen is identifiable in any government view or export — proven by a re-identification test	P1 + P2
7	Test coverage ≥ 80% on new code; zero open critical or high-priority bugs	P4
8	Load test: 95th percentile under 300 ms at 10,000 requests/second	P4
9	Zero high or critical security findings survive	P4
10	Full journey passes a screen-reader walkthrough (WCAG 2.1 AA)	P2
11	Disaster-recovery drill: restore into a clean region, recovery time under 4 hours	P4
12	Rollback of any service completes in under 5 minutes — proven by drill	P4
13	Every metric has exactly one definition; no two dashboards disagree	P3
14	Every alert has a runbook a new person can follow unaided	P4

4. Weekly Working Rules

When	What	Duration
Daily 9:45	Standup — <i>yesterday / today / blocked</i> . Hard stop.	12 min
Daily 16:30	Pull request sweep — every open PR reviewed or explicitly deferred	30 min
Daily 18:00	Build must be green. Whoever broke it fixes or reverts it.	—
Monday	Sprint planning (2-week sprints)	60 min
Thursday	Risk + security review	30 min
Friday	Demo of working software (recorded) + retrospective	45 min

Rules that are not negotiable 1. Branch names: `feat/r2-p1-booking-state-machine` — *type / review / owner / summary*. Maximum 3 days alive. 2. Commits: `feat(booking): reject illegal state transitions` — type, scope, imperative summary. 3. Every pull request needs **1 approval**; **2 approvals** for database migrations, authentication, payments and the emergency service. 4. Nobody merges their own code. 5. Blocked for more than 2 hours? Say so publicly. Sitting silently on a blocker is the only genuinely unacceptable behaviour on this team.

Definition of Done — code merged · tests written (≥ 80%) · works on the low-end reference phone · has loading/empty/error/offline states · documented · demoed on Friday.

5. If You Fall Behind — Cut In This Order

Decide this now, not under pressure the week before a review.

Order	Cut	Days freed	Cost
1	Android Auto prototype	2	Zero — it was always a bonus
2	USSD (keep SMS + IVR)	3	Low — SMS and IVR already cover feature phones

Order	Cut	Days freed	Cost
3	Sound-based diagnosis	2	Low — it is already flagged as a spike
4	Fleet dashboard	4	Medium — reduces the business story
5	3 of the 8 languages	3	Medium — keep Hindi, English, Tamil, Telugu, Marathi
6	Predictive maintenance model	4	Medium — rules-based version still ships

Never cut, under any circumstance: the emergency path, the offline engine, the SMS fallback, or security testing. These are the four things that make this a platform rather than an app.

6. Top Risks

Risk	Who	What we do about it
TRAI DLT approval for SMS takes weeks	P4	Started in Week 1 , before anything else. Two gateway vendors registered.
No real training data at the start	P3	Every AI feature ships rule-based in Review 2. Models are an upgrade, never a dependency.
Offline sync corrupts booking data	P1 + P2	Conflict matrix written <i>before</i> coding; 30-scenario test suite; server is always authoritative on booking state.
Crash detection falsely calls emergency services	P3 + P1	Model raises a <i>signal</i> , never an incident. 30-second cancel window. Call-back verification. Auto-dispatch blocked in code.
P2 overloaded in Review 2	P2	Admin + Fleet portals moved to P4; Android Auto is the declared first cut.
Team of 4 undersized for the scope	All	Reviews are milestone-gated, features are flag-controlled, and the cut list above is agreed in advance.

P1 — Backend & Database Lead

RoadAssist Bharat · 87 days across 3 reviews Owns: server services · PostgreSQL/PostGIS schema · all API contracts · authentication & RBAC · booking and dispatch engine · emergency core · payments · sync server. Backs up **P3**.

Your three standing rules 1. The API specification is generated from code and published on every merge. It is the single source of truth. 2. Nobody is ever blocked waiting on a contract from you. Specification and mock first, implementation second. 3. Every database migration is reviewed by a second person and has a tested rollback.

REVIEW 1 — Foundation · Weeks 1–5 · 25 days

Week	Task	Days
1	Integration feasibility: VAHAN/SARATHI, ERSS 112, UPI/NPCI, FASTag. Written go/no-go and a fallback for each.	2
1–2	Candidate entity list (~55 entities with cardinalities) from mechanic and fleet interviews	2
2	Backend backlog (~70 stories with acceptance criteria) + estimates; monorepo scaffold with CI	4
3	C4 architecture diagrams + 15 ADRs ; event catalogue; API style guide	5
3–4	60-table schema across 13 modules; ER diagram; index plan with query evidence	4
4	Audit log triggers; soft delete; optimistic-locking version columns; partitioning; seed generator (100k rows)	2
5	Authentication : OTP flow, JWT + rotating refresh with reuse detection, device binding	3
5	RBAC policy engine (7 roles × 45 permissions) + consent ledger (DPDP-compliant)	3

Deliverables

15 ADRs · C4 diagram set · API style guide · 60-table schema with migrations · ER diagram · seed generator · auth service · RBAC engine · consent ledger · TypeScript auth SDK.

Your Review 1 demo — 6 minutes

Run migrations on an empty database → seed 100,000 rows → run a geospatial "nearest mechanic" query showing **under 50 ms with the index and over a second without**. Update a record and show the audit trigger capturing before/after. Then log in by OTP on a real phone, steal the refresh token, replay it — and watch the entire session family get revoked and the user notified.

Passed when

Migrations roll back cleanly twice · every table has PK, timestamps, soft delete, version and an owner · zero sequential scans on the top-20 queries · OTP brute force blocked at 5 attempts per 15 minutes · 20 RBAC tests pass including 6 negative access-control cases.

REVIEW 2 — Core Product · Weeks 6–12 · 33 days

Week	Task	Days
6–8	140 API endpoints across 12 modules — users, vehicles, bookings, dispatch, mechanics, pricing, payments, notifications, fleet, reviews, support, webhooks	15
6–7	<i>(within the above)</i> Booking state machine with guarded transitions; dispatch scoring and offer ladder; pricing engine with transparent breakdown	—
8	<i>(within the above)</i> UPI payments, refunds, subscriptions; transactional outbox for reliable events	—
9	AI client wrapper: 800 ms timeout, circuit breaker, fallback registry , prediction logging	3
9	Feature pipeline into the feature store for P3	—
10	Composite BFF endpoints so the home screen is one request, not six (this matters enormously on 2G)	3
11–12	Sync server : cursor-based change feed, per-field conflict resolution, tombstones, batch replay	6

Week	Task	Days
12	Geofencing, live tracking channel, highway kilometre-marker resolution	3
12	Diagnostics service + DTC knowledge base (~5,000 codes) + mechanic pre-brief	3

Deliverables

12 service modules · 140 endpoints · OpenAPI 3.1 specification · generated TypeScript client · contract tests · booking state machine · sync server · **conflict resolution matrix** · geo services · diagnostics service.

Your Review 2 demo — 8 minutes

Scripted end to end: user requests help → 3 mechanics offered → one accepts → status moves to completion → invoice → UPI payment → review → partner webhook fires. **Then break it deliberately:** cancel mid-flight, time out a mechanic, fail a payment. Every path handled. Finally, kill the AI service completely and run the whole booking again on fallbacks.

Passed when

100% of the v1 API implemented · every write endpoint idempotent under replay · the state machine rejects every illegal transition · booking still completes with AI switched off · 30-scenario conflict suite passes.

REVIEW 3 — Complete System · Weeks 13–18 · 29 days

Week	Task	Days
13–14	Emergency engine — separate deployable, own database pool, own quota, spread across 3 zones	6
13	(within) Escalation ladder: contacts → responders → ambulance → ERSS 112 handoff with human confirmation	—
14	(within) Break-glass medical access with full audit; degraded SMS path sharing zero dependencies with the main platform	—
15	Government service: k-anonymity ≥ 10 enforced at the query layer , jurisdiction isolation, officer audit log	3
15	CDC pipeline Postgres → Kafka → ClickHouse with PII stripping	3
16	Property-based tests (state machine, pricing) + mutation testing; fix load and security findings	5
16	Verify security fixes from P4's penetration test	2
17	Online migrations (no locking), health probes, graceful shutdown, service runbooks	3
17–18	Query tuning from live statistics, caching layer, payload reduction, connection pooling	4
18	Scaling plan to 10M users with named bottlenecks; technical debt register	3

Deliverables

Emergency service (isolated) · 112 handoff adapter · break-glass audit · degraded SMS path · government service with anonymisation · CDC pipeline · property + mutation tests · service runbooks · scaling plan · debt register.

Your Review 3 demo — 7 minutes

Simulate a crash on a real device. Countdown → contacts get an SMS with a live location link → nearest responder notified → incident room opens, **under 10 seconds**. Then scale the entire rest of the platform to zero replicas and do it again — it still works. Finally, play attacker against the government API: try to identify one specific citizen, fail every time, and show the audit log that flagged you.

Passed when

Crash to responder under 10 seconds · emergency survives total platform failure · AI cannot auto-dispatch (enforced in code) · re-identification test fails on all 20 attack strategies · deploy a schema change to a 10-million-row table with zero downtime.

Your Key Design Decisions (defend these to the panel)

Decision	Why
Modular monolith, not microservices	Four people cannot operate 15 independently deployed services. Module boundaries are strict and split-ready. Emergency is the one genuine separate deployable — it needs a different reliability class.
Emergency deployed separately	Life safety needs its own failure domain. A bug in bookings must never be able to kill an SOS.
Server always wins on booking state	An offline client can never force an illegal state. State machine integrity is not negotiable.
AI is never on the correctness path	Every AI call has a deterministic fallback. Models advise; rules decide.
Money in integer paise, never floats	Floating-point arithmetic on currency produces reconciliation failures that are very expensive to find later.
Conflict matrix written before coding	The top technical risk in the project. Field-level rules are decided and reviewed before the first line of sync code.

P2 — Frontend & Mobile Lead

RoadAssist Bharat · 86 days across 3 reviews Owns: React Native app · web, government, admin and fleet portals · design system · offline client · maps UI · accessibility · 8 languages. Backs up **P4**.

Your three standing rules 1. The **reference device is the truth**: Android 10, 2 GB RAM, throttled 3G. If it does not work there, it does not work. 2. Every screen ships with **four** states built — loading, empty, error, offline. A happy path alone is not done. 3. No hardcoded user-facing text, ever. Eight languages from the first commit.

REVIEW 1 — Foundation · Weeks 1–5 · 17 days

Week	Task	Days
1	Device and connectivity study — measure real 2G/3G latency on 3 highway corridors; catalogue the Indian device mix	2
1–2	20 user interviews (UX focus); competitor teardown; accessibility and literacy findings	2
2	6 personas + 4 journey maps : highway breakdown, city no-start, night accident, fleet truck failure	2
2	Information architecture, navigation map, component inventory	2
3	Low-fidelity wireframes for the 12 critical screens with all four states	3
3–4	Design system foundations: colour, type, spacing, motion, touch targets; app scaffolds	3
4	Local offline database schema and storage budget (≤ 120 MB, with eviction)	1
5	Auth screens: phone entry, OTP with SMS auto-read, profile setup, consent centre	2
5	Secure token storage (Keychain/Keystore), silent refresh with request queueing, biometric gate	—

Deliverables

Device matrix · connectivity report · 6 personas · 4 journey maps · information architecture · wireframes for 12 screens · design system foundations · auth screens · secure storage · clickable prototype.

Your Review 1 demo — 5 minutes

Hold up the **reference device physically**. Show a competitor's app struggling on it — that is the brief. Then walk the clickable prototype of the complete breakdown journey in Hindi. Finish by logging in over throttled 3G with SMS auto-read, then killing the app mid-refresh to show it recovering cleanly.

Passed when

Wireframes cover loading, empty, error and offline for all 12 screens · design system renders in all 8 languages with no truncation · tokens stored only in Keychain/Keystore, never in plain storage · 10 concurrent 401s trigger exactly **one** refresh.

Your design constraints (evidence-based, from Week 1)

Works one-handed, in rain, at night, on a highway shoulder · readable in direct sunlight (contrast $\geq 7:1$) · comprehensible at low literacy (icon + colour + voice, never text alone) · never a blank screen · never a silent failure · $\text{APK} \leq 28 \text{ MB}$ · $\leq 30 \text{ MB}$ data per month.

REVIEW 2 — Core Product · Weeks 6–12 · 36 days

Week	Task	Days
6–7	Design system package — 42 components, dark + light, 8 languages, motion system	6
7–8	Citizen app: onboarding, garage, request-help flow , diagnosis, matching, live tracking, payment, history	12
9	Mechanic app: job feed, accept, navigation, checklist, parts, invoice builder, earnings	4
9	Web PWA parity for the core flows	2

Week	Task	Days
10	AI confidence UI + user feedback capture that feeds P3's training loop	2
11–12	Offline client — local database, priority operation queue, sync engine, conflict resolution screen, background sync	10
12	(within) <code>useOfflineQuery</code> as the only data-access pattern in the app — this is what makes offline uniform rather than per-screen	—

Note: Admin console and Fleet dashboard moved to P4 to keep this review achievable. Android Auto is the declared first cut if you slip.

Deliverables

Design system (42 components) · citizen app (Android + iOS) · mechanic app · web PWA · offline client library · sync status and conflict UI · Storybook · visual regression suite.

Your Review 2 demo — 8 minutes

The full journey on a real **₹8,000 Android phone**, throttled to 3G, in Hindi, one-handed, at maximum brightness to simulate sunlight. **No simulator**. Then switch to airplane mode: add a vehicle, run an on-device diagnosis, create a help request, view an offline map, cancel and re-create. Reconnect — everything syncs and one deliberate conflict is resolved in a screen that makes the choice obvious.

Passed when

Full journey works on Android 10 / 2 GB / 3G · every screen has all four states · airplane-mode journey syncs with zero data loss · cold start under 2.5 seconds on the reference device · optimistic writes visibly marked "pending" until confirmed — never a silent revert.

REVIEW 3 — Complete System · Weeks 13–18 · 33 days

Week	Task	Days
13	Maps UI: MapLibre components, offline map packs with resumable download, live tracking, landmark location picker	7
13	(within) Diagnosis UI: OBD Bluetooth pairing, guided photo capture, drivability verdict	—
14	SOS UI — long-press button, full-screen crash countdown over the lock screen, incident room	4
14–15	Government portal — live incident map (10,000 markers, no jank), blackspot analysis, 6 statutory report generators	6
15	Analytics dashboards; fleet and mechanic earnings screens	4
16	End-to-end tests (Detox + Playwright) for 12 journeys; visual regression baseline	4
16	Device farm across 12 Android + 4 iOS; accessibility audit; 8-language QA; bug fixes	3
17	App store submission with staged rollout; over-the-air update channel	3
17–18	Optimisation — APK size, startup, jank at 60fps, memory leaks, data usage	2

Deliverables

Maps components + offline packs · diagnosis flow · SOS and incident room · government portal · analytics dashboards · end-to-end test suites · device compatibility matrix · accessibility report · published apps · performance report.

Your Review 3 demo — 7 minutes

Download a district map pack over WiFi, go to airplane mode, search a landmark and get directions — fully offline. Then trigger a simulated crash **with the phone locked and in a pocket**: the countdown takes over the screen with alarm and vibration. Then the government portal: find a blackspot, generate the statutory PDF. Close with the killer demo — **the entire booking journey driven by a screen reader with the display switched off**. If a blind user can call for roadside assistance, the app is well built.

Passed when

Offline navigation works with the device in airplane mode · countdown renders over the lock screen and in Do Not Disturb · government map handles 10,000 incidents without jank · WCAG 2.1 AA passes a real screen-reader walkthrough (not just an automated scan) · APK ≤ 28 MB · cold start ≤ 2.0 seconds · ≤ 30 MB data per month.

Your Key Design Decisions (defend these to the panel)

Decision	Why
One shared component library across 5 apps	It is the only way one person ships five clients in 18 weeks.
Three portals, one Next.js app	Government, admin and fleet share a codebase with role-based routing instead of being three projects.
System fonts, no custom font download	Saves ~400 KB and renders all 8 Indian scripts correctly. Custom fonts routinely break Devanagari and Tamil.
FlashList everywhere, never FlatList	Long lists are where low-end Android devices actually fall over.
<code>useOfflineQuery</code> as the single data pattern	Offline becomes a property of the architecture rather than something re-implemented per screen.
Confidence shown as words, not percentages	"We're fairly sure" versus "this is a guess — a mechanic will confirm." Users cannot calibrate a 0.73.
Animation auto-disabled on low-end devices	Detected at runtime. Motion is a luxury; responsiveness is not.

P3 — AI & Data Lead

RoadAssist Bharat · 70 days across 3 reviews Owns: 9 AI subsystems · model training and serving · on-device inference · feature store · voice and NLU · analytics pipeline. Backs up **P1**.

Your three standing rules 1. **Record the baseline before you train.** A model with no documented baseline cannot be said to work. 2. Every model ships with a **model card** and a **fallback**. No exceptions, including for models you are proud of. 3. No model is ever on the critical path for correctness or safety. **Models advise; rules decide.**

Your total is lower than the others on purpose. Model work is research-shaped and estimates are less reliable — the slack absorbs the experiments that fail, and some will.

The 9 AI Systems

#	System	Input → Output	Method	Runs on	Baseline to beat
1	Vehicle Diagnosis	Symptoms, DTC codes, history → ranked causes + severity + parts	Retrieval + gradient boosting; LLM only writes the explanation	Server + device	Rules on DTC lookup: top-3 = 0.62
2	Image Diagnosis	Photos → damage type, part, severity	MobileViT / EfficientNetV2, multi-label + severity head	Server + device	Non-expert labelling: top-3 = 0.55
3	Voice Assistant	Speech (8 languages) → intent + slots	Whisper fine-tuned on Indian speech → MuRIL intent classifier	Server; keyword on device	Keyword matching: 0.48
4	Predictive Maintenance	Telemetry + history → failure probability at 30/90 days	Survival analysis + LightGBM per component	Server, nightly	Fixed service interval: recall = 0.31
5	Mechanic Matching	Request + mechanic features → ranked list	LambdaMART ranker + separate ETA regressor	Server, <300 ms	Distance + rating sort: 0.41
6	Crash Detection	Accelerometer, gyroscope, GPS at 100 Hz → crash probability	1D CNN + GRU, INT8 quantised, ~1.5 MB	Device only	Peak-G threshold: 1 false alarm per 12 hours
7	Fraud Detection	Booking graph, payments, GPS traces → risk score + reasons	Graph features + isolation forest + gradient boosting	Server, async	Manual rules: precision 0.34
8	Government Analytics	Incidents, weather, road geometry → risk forecast, blackspots	Spatial clustering + temporal fusion model	Server, batch	Historical average by location
9	Offline AI Bundle	Symptoms, sensors → diagnosis + crash detection with no signal	Quantised and distilled versions of 1, 3 and 6	Device, ≤ 15 MB	The alternative is nothing

REVIEW 1 — Foundation · Weeks 1–5 · 13 days

Week	Task	Days
1	Data availability audit — what exists, what can be acquired, what must be synthetic	2
1–2	Feasibility spikes: crash detection from public sensor data; speech recognition on Indian-accented audio	2
2	Honest classification of all 9 systems: trainable now / trainable later / rules-only in v1	1
2	AI backlog (~35 stories), labelling plan and cost, evaluation protocol with baselines recorded	2
3	AI Gateway design: capability routing by model version, shadow mode, guardrails, fallback contract with P1	3
3–4	Feature store schema + label capture design (this is the highest-leverage decision of the whole review)	2
5	Fraud signal catalogue handed to P1; privacy-respecting device fingerprinting design	1

Deliverables

Data audit · feasibility report with the 9-system classification · labelling plan · evaluation protocol with recorded baselines · AI Gateway design · feature store schema · label taxonomy · model card template.

Your Review 1 demo — 5 minutes

Present the **honest classification**: which of the 9 systems are genuinely models in v1, which are rules wearing an AI label, and exactly what data would change that. Then run an end-to-end toy pipeline — data → train → registry → export → serve → prediction. The plumbing works before any real model exists. *This candour is what makes everything you claim later credible.*

Passed when

Every one of the 9 systems is classified honestly · every model has a **baseline recorded before training begins** · label capture is designed even though it will not be used for months · the training/serving feature parity test exists and fails when a feature is changed in only one place.

REVIEW 2 — Core Product · Weeks 6–12 · 33 days

Week	Task	Days
6–7	AI Gateway live with rule-based logic for all 8 capabilities — so P1 and P2 integrate on day one, months before any model exists	4
7	AI presentation contract with P2: confidence bands as plain language , not percentages	1
8–9	Vehicle Diagnosis model — retrieval + re-ranker + constrained LLM explanation	4
9	Image Diagnosis model — with augmentation for rain, night, motion blur, low-end cameras	3
9–10	Voice Assistant — speech recognition fine-tuning, intent + slots, code-switched Hinglish	3
10	Predictive Maintenance — survival analysis per component	3
10	Mechanic Matching — ranker + ETA regressor with an explicit fairness constraint	2
11	Crash Detection — the highest-stakes model in the project	3
11	Fraud Detection — graph features + anomaly detection	2
12	Model registry, shadow mode, one-config rollback	2
12	On-device bundle ≤ 15 MB — quantisation, distillation, benchmarking on 5 real devices	4
12	Vehicle-class rule packs (two-wheeler, auto, tractor) + EV diagnostics	2

Deliverables

AI Gateway with rules **and** models · 7 trained models · 7 model cards · evaluation harness · on-device bundle ≤ 15 MB · rule packs per vehicle class · EV diagnostics module.

Your Review 2 demo — 8 minutes

For each model: **the baseline, the model, the delta, and the failure modes**. Then a live diagnosis — photograph a real damaged part, get the fault, severity, parts list and a "do not drive" verdict in Hindi. Then unplug the models entirely and show the rules fallback keeping the product working. Finish in **airplane mode on a ₹8,000 phone**: describe a symptom, get a diagnosis in under 200 ms, nothing touching the network.

Passed when

Every model beats its recorded baseline on a held-out set · on-device inference under 120 ms · on-device bundle ≤ 15 MB · **measured false "safe to drive" rate is zero** · every model has a confidence threshold and a working fallback · quantisation accuracy loss reported, not assumed.

Rule packs are not a shortcut. Most Indian two-wheelers have no OBD port. For those vehicles, structured symptom questioning built with real mechanics is the *correct* engineering choice, not a fallback.

REVIEW 3 — Complete System · Weeks 13–18 · 24 days

Week	Task	Days
13	Crash detection to production quality — threshold tuning against real dogfooding data; severity model	3

Week	Task	Days
14	Blackspot detection (spatial clustering with significance testing); incident forecasting for government	3
15–16	Analytics — event taxonomy, ClickHouse schema, metric dictionary , anomaly alerting	5
16	Bias audit across all models — performance by vehicle class, language, region and device tier	2
16	Adversarial testing: prompt injection, adversarial images, sensor spoofing, GPS spoofing	2
17	Model serving in production; promotion pipeline with automatic rollback on metric regression	3
17–18	Inference optimisation: batching, caching, distillation, cost per million inferences	3
18	AI roadmap and the data flywheel — which model improves first, at what data volume	3

Deliverables

Production crash detection with measured false-positive rate · blackspot + forecasting models · ClickHouse analytics · metric dictionary · 9 finalised model cards with bias audits · model serving with auto-rollback · AI roadmap.

Your Review 3 demo — 6 minutes

Present the **real false-positive number** from 200 hours of driving. Then drive over a speed breaker on camera — nothing happens. That is the demo. Then run blackspot detection on a state's historical data and overlay the official blackspot list — show the overlap **and the new candidates you found**. Close by promoting a deliberately bad model and watching it roll back automatically in under 2 minutes.

Passed when

Crash false-positive rate ≤ 1 per 1,000 driving hours over **real** driving · the model raises a signal and can never auto-dispatch · every model has a bias audit · every metric has exactly one definition · a bad model rolls back automatically.

The pre-committed decision on crash detection. If the measured false-positive rate over ≥ 200 real driving hours does not meet the target, crash detection **ships disabled by default** and manual SOS carries the feature. This is decided now so it is not a judgement call under demo-day pressure.

Your Key Design Decisions (defend these to the panel)

Decision	Why
Rules ship first, models replace them	All 8 AI endpoints go live in Review 2 as rules. Models then upgrade them behind an unchanged contract. AI is never a blocker, and the rules remain the permanent production fallback.
The LLM never chooses the diagnosis	It explains a structured result selected by a classifier. This removes hallucination from the safety path entirely.
Crash detection runs on-device only	It must work with no connectivity — which is exactly the condition of a highway crash.
The matching model has a fairness constraint	A ranker optimised purely for acceptance concentrates all jobs on a few mechanics and destroys the marketplace. Job distribution is tracked as a first-class metric.
Fraud detection never auto-suspends	It queues for human review with mandatory reason codes. A false accusation destroys a mechanic's livelihood.
Precision over recall on maintenance alerts	A user who ignores three false warnings will ignore the real one.
Minimal MLOps stack	Tracking, versioning and plain containers. No orchestration platform — one person cannot operate that.

P4 — DevOps, QA & Security Lead

RoadAssist Bharat · 76 days across 3 reviews Owns: cloud infrastructure · CI/CD · observability · all testing · security · SMS/IVR/USSD telecom gateway · deployment · compliance. Backs up **P2**.

Your three standing rules 1. The build is never broken on the main branch for more than 30 minutes. You own that number. 2. You are the only person who can say **"this is not ready to ship."** Use it. 3. Every alert has a runbook **before** it is switched on. An alert without a runbook trains people to ignore alerts.

REVIEW 1 — Foundation · Weeks 1–5 · 17 days

Week	Task	Days
1	Start TRAI DLT registration for SMS templates — do this before anything else. It is the longest lead item in the entire project and nobody can compress it. Register more templates than you think you need, with two vendors.	1
1	Cloud provider evaluation (data-localisation compliant); telecom gateway comparison; cost model	2
1–2	Compliance obligation register — DPDP Act 2023, TRAI, cloud empanelment, payment regulations — each mapped to the phase that satisfies it and the evidence that proves it	1
2	CI/CD skeleton: lint, typecheck, test, build, security scan on every pull request	2
2	Branch protection, CODEOWNERS, secret scanning, dependency automation	1
2	Test strategy document; environment strategy (local → preview → dev → staging → production)	1
3	STRIDE threat model with P1 across all 9 trust boundaries → a control list mapped to phases	4
3	SLO definitions with error budgets (Emergency gets 99.99%, everything else 99.9% or lower)	—
4	Database infrastructure, point-in-time recovery, nightly automated restore verification	2
5	Secrets management with rotation; SMS gateway with two vendors and automatic failover ; edge rate limiting; WAF	3

Deliverables

DLT registration submitted · compliance register · cost model · CI/CD pipeline · branch protections · test strategy · threat model · SLOs with error budgets · database infrastructure with verified backups · secrets management · dual SMS gateway.

Your Review 1 demo — 5 minutes

Open a pull request containing a lint error, a failing test **and a hardcoded password**. Watch CI block all three in under 8 minutes. Then delete a table in a scratch environment and restore it from point-in-time recovery to five minutes before the deletion. Finish by disabling the primary SMS vendor mid-login — the OTP still arrives via the secondary within the same latency budget.

Passed when

CI blocks lint, tests and secrets in under 8 minutes · every regulatory obligation is mapped to a phase and an evidence artefact · a backup is restored and verified automatically, nightly · SMS failover works, not just configured · every one of the top 10 threats has a named control and the phase it lands in.

A backup that has never been restored is not a backup. That is why the restore runs nightly from Review 1, not once at the end.

REVIEW 2 — Core Product · Weeks 6–12 · 27 days

Week	Task	Days
6–7	Observability stack — metrics, logs, distributed tracing, error reporting; auto-generated dashboard per service	4
7	<i>(within)</i> Log scrubbing at the library level — PII, tokens and payment data redacted automatically, verified by a test that logs a fake token	—

Week	Task	Days
7	Integration test infrastructure with real databases in containers; ephemeral preview environment per pull request	—
8	Mobile CI: signed Android + iOS builds distributed to the team on every merge, so everyone dogfoods continuously	3
9	ML infrastructure for P3; training on cheap interruptible compute; GPU scale-to-zero and budget alerts	3
10	Admin console + Fleet dashboard (taken off P2 to keep Review 2 achievable)	6
11–12	Telecom gateway — SMS command grammar, IVR menu tree in 8 languages, USSD session flow, vendor abstraction	6
12	Self-hosted map tile server and routing engine; offline map pack build pipeline	3
12	Network chaos harness for P2's offline testing; media upload pipeline with EXIF stripping and virus scanning	2

Deliverables

Observability stack · log scrubbing · preview environments · mobile CI with signed builds · ML infrastructure · admin + fleet portals · **telecom gateway (SMS, IVR, USSD)** · tile and routing infrastructure · chaos harness.

Your Review 2 demo — 7 minutes

Follow **one request** from the mobile app through the gateway, three services, the database and the AI gateway — as a single distributed trace. Show the log-scrubbing test catching a leaked token. Then the headline: on an actual **₹1,200 feature phone**, send **MADAD** by SMS, receive a reply, confirm location, get the mechanic's name and number, cancel. Then call the IVR number and do the same in Tamil. **No smartphone anywhere in that demo.**

SMS command grammar

Command	Also accepts	Effect
HELP	MADAD , SAHAYA , H	Start an assistance request
SOS	EMERGENCY , 112	Emergency — highest priority, immediate escalation
STATUS	S	Booking status and mechanic ETA
CANCEL	C	Cancel the active booking (confirms first)
LANG HI	any language code	Switch language for all future messages

Unrecognised input replies with the command list, never an error. A user who mistypes must not be stuck.

Passed when

One request produces one complete distributed trace · a fake token in a log is provably redacted · a feature-phone user can request help, track and cancel entirely over SMS · IVR works in all 8 languages with keypad input always available as a fallback · GPU pool scales to zero when idle.

REVIEW 3 — Complete System · Weeks 13–18 · 32 days

Week	Task	Days
13	Isolated emergency infrastructure — own node pool, own database pool, own SMS quota, spread across 3 zones	4
13	(within) The chaos test that matters most: scale every non-emergency service to zero, sever the main database — SOS must still reach a responder	—
14	Government mTLS: certificate issuance, renewal, revocation; tamper-evident audit log with hash chaining	3
14	ClickHouse cluster; change-data-capture infrastructure with replication-lag alerting and a kill switch	3
15–16	Load testing — 10k requests/sec steady, 50k spike, 100k concurrent tracking connections, 24-hour soak	4
15–16	Security testing — full OWASP checklist, dynamic + static analysis, dependency and container scanning, red-team pass	3
16	Chaos experiments — kill pods, partition the network, exhaust connections, expire certificates	3

Week	Task	Days
17–18	Production deployment — infrastructure as code, GitOps, canary releases with automatic rollback	8
17–18	(within) Disaster-recovery drill : restore into a clean region, measure recovery time and data loss against targets	—
17–18	(within) 25+ runbooks , one per alert; on-call handbook	—
18	Infrastructure right-sizing, autoscaling tuning, cost model per 100k users; compliance roadmap	4

Deliverables

Isolated emergency infrastructure with chaos proof · government mTLS and audit log · analytics infrastructure · load, security and chaos test reports · production infrastructure as code · disaster-recovery plan **with a passed drill** · 25+ runbooks · cost model · go/no-go recommendation.

Your Review 3 demo — 8 minutes

Scale the **entire platform to zero replicas**, live, in front of the panel. Trigger an SOS from a phone. It still works. Then deploy a change through the full pipeline to production with one approval click; deliberately break the canary and watch it roll back automatically. Then run the disaster-recovery drill: restore the platform into a clean region, timed, live. Finish by presenting the load and security reports — **including what failed and what you did about it**.

Passed when

Emergency survives with every other service down · 95th percentile under 300 ms at 10,000 requests/sec · graceful degradation at 50k spike, not collapse · zero high or critical security findings survive · rollback of any service under 5 minutes, proven by drill · recovery time under 4 hours, data loss under 5 minutes, **proven not asserted** · a person who did not write a runbook can resolve a simulated incident using only that runbook.

Chaos experiments (each must end in documented degraded behaviour, never an outage)

Experiment	Expected
Kill 1 of 3 API pods	No user-visible impact
Kill all AI pods	Rules fallback serves everything; no errors reach users
Kill the primary database	Failover under 60 seconds; in-flight requests fail cleanly
Exhaust the connection pool	Requests queue then shed with a retry hint; no crash
Scale everything except Emergency to zero	SOS still works via the degraded path
Primary SMS vendor total outage	Automatic failover, no user impact
Clock skew of 5 minutes on one node	Token validation still correct within tolerance

Your Key Design Decisions (defend these to the panel)

Decision	Why
DLT registration started in Week 1	It is the longest lead item in the project and blocks all authentication. Everything else can be parallelised; this cannot.
Two SMS vendors from day one	The SMS gateway is a single point of failure for all authentication. One vendor is not a design.
Emergency gets its own failure domain	Life safety needs a different reliability class: separate deploy, separate quota, and a degraded path sharing zero dependencies with the platform.
Deliberately boring infrastructure	Managed services, one cluster per environment, no service mesh. A four-person team must be able to operate this at 3 a.m.
Self-hosted maps	Commercial per-tile licensing at national scale is prohibitive. Tiles become a compute cost, not a licence fee.

Decision	Why
Log scrubbing in the library, not by convention	Conventions fail under deadline pressure. Automatic redaction does not.
Error budget policy is automatic	Burn 50% of a service's budget and feature work on it stops. It is a rule, not a discussion.